

Developer Burnout Risk Classifier

A multi-class Random Forest for proactive engineering-team triage

Ian Menachery | ECON 3916: Statistical & Machine Learning for Economics | Spring 2026

1. Problem Statement

Most engineering orgs find out about burnout too late. By the time a senior developer gives notice or goes on extended leave, the replacement search and the loss of institutional knowledge are already costs you can't walk back. Industry estimates for replacing a mid-level engineer tend to land somewhere in the range of 1.5x to 2x annual salary. The question this project tries to answer is whether observable work patterns, plus a single self-reported stress score, can classify a developer into a Low, Medium, or High burnout tier accurately enough to be useful for weekly triage.

I want to be clear up front that this is a prediction problem, not a causal one. The goal isn't to prove that extra caffeine or a missed workout is what causes burnout. The goal is to learn a mapping from observable inputs to an at-risk label that's accurate enough to surface developers who should get a human check-in. That distinction matters once you try to deploy something. A predictive classifier can justify screening, but it can't justify intervention design.

The stakeholder I have in mind is an engineering team lead or an internal wellness coordinator running a weekly 1:1 cadence across somewhere between ten and thirty reports. The decision the model actually supports is a short-list problem: which of those reports get a non-routine workload conversation this cycle. Without a model, that call gets made on gut feel plus visible signals (someone skipping standup, missing deadlines), and those signals tend to show up only after burnout is already advanced. With a reliable High flag that fires earlier, the team lead can reassign tickets, block off a no-meeting day, or push some PTO before attrition becomes the first visible symptom. The predicted tier is an input to that conversation. It's not a substitute for the conversation, and it absolutely should not feed into performance reviews, comp, or any other adverse employment action.

2. Data Description

I used the publicly available Developer Burnout Prediction Dataset on Kaggle (asifxzaman, accessed April 15, 2026). It has $N = 7,000$ developers, 11 input features, and a continuous Burn Rate target in $[0, 1]$. The features break down into three groups. Demographics: age and experience_years. Work patterns: daily_work_hours, screen_time, meetings_per_day, commits_per_day, and bugs_per_day. Lifestyle and self-report: sleep_hours, exercise_hours, caffeine_intake, and stress_level on a 1 to 10 scale. The continuous Burn Rate was bucketed into three tiers (Low, Medium, High) using cutoffs at roughly the 33rd and 67th percentiles. Those cutoffs are a modeling choice I made, not a ground-truth definition, and I flag this as a downstream limitation later in the report.

The data looks clean on standard diagnostics, and that's actually part of what concerned me. Missingness is a uniform 2% across every single column, which is the textbook pattern for MCAR (missing completely at random). I dropped rows where the target label itself was missing to keep the labels clean, and filled in missing feature values with the column median. I picked median over mean because a couple of the features (bugs_per_day and caffeine_intake) are mildly right-skewed. Tukey fences (1.5 times the IQR) flagged exactly zero outliers on any of the key continuous features. The target is imbalanced: Medium dominates, and Low and High are substantial minorities. That imbalance is why I don't rely on accuracy as a headline metric later on and go with macro F1 instead.

Two things about this data profile matter for how you should read the results. First, perfectly uniform 2% missingness and zero Tukey outliers across 7,000 rows of behavioral data is not what you see in the wild. Together, those point pretty strongly toward the dataset being either synthetic or heavily pre-cleaned. Second, because this is a public Kaggle dataset rather than data pulled from a specific company, anything I train on it has limited external validity. The performance numbers in Section 4 describe how the model does on this distribution. They don't describe how it would do on a real engineering org, and my Recommendation section treats closing that gap as a required next step rather than an optional one.

3. Methodology

After imputation, I split the cleaned dataset 80/20 into training and held-out test sets using `train_test_split` with `stratify=y` (so the tier proportions match across splits) and `random_state=42` for reproducibility. Every downstream step uses the same seed, so the numbers below are reproducible from the notebook exactly as written.

I compared two models. The baseline is a multinomial Logistic Regression with `class_weight='balanced'` and `max_iter=2000`. I picked it because it's the natural linear reference point, and the coefficients are directly interpretable. The candidate model is a Random Forest with 200 trees and the same balanced class weighting. I went with Random Forest over XGBoost or LightGBM for a few reasons. It handles multi-class classification natively without wrapping in one-vs-rest. It gives you per-feature Gini importances that are easy to communicate to a non-technical stakeholder. And honestly, for a class project where the goal is a defensible screening heuristic rather than a leaderboard score, the small extra accuracy you might squeeze out of boosting isn't worth losing the interpretability.

For evaluation I ran 5-fold cross-validation on the training split using macro F1 as the scoring metric. Macro F1 weights the three tiers equally, which matters given the class imbalance. If I used plain accuracy, you could get a high score just by predicting Medium on everyone, but that would tank macro F1 because the Low and High F1 components would collapse. I set hyperparameters deliberately rather than doing a full grid search. 200 trees is well past the point where Random Forest error flattens out, and I kept default tree depth because shallower trees didn't improve out-of-bag error. I considered SMOTE-style oversampling for the minority classes but chose not to use it. Adding synthetic rows to a dataset that already looks synthetic seemed more likely to bake in whatever artifacts the source data has than to actually help generalization.

4. Results

The Random Forest beats the baseline on both the mean CV macro F1 and its spread across folds (Table 1, Figure 1). The baseline logistic regression gets macro F1 = 0.945 with a standard deviation of 0.006. The Random Forest gets 0.991 with a standard deviation of 0.002. The 4.6-point gap is wider than three standard deviations of either estimate, so this isn't a fold-selection fluke. The Random Forest's std is also about a third of the baseline's, which tells me it generalizes more consistently across folds, not just better on average.

Model	CV macro F1 (mean)	CV macro F1 (std)	Folds
Logistic Regression (baseline)	0.945	0.006	5
Random Forest (selected)	0.991	0.002	5

Table 1. Five-fold CV macro F1 on the training split, random_state=42.

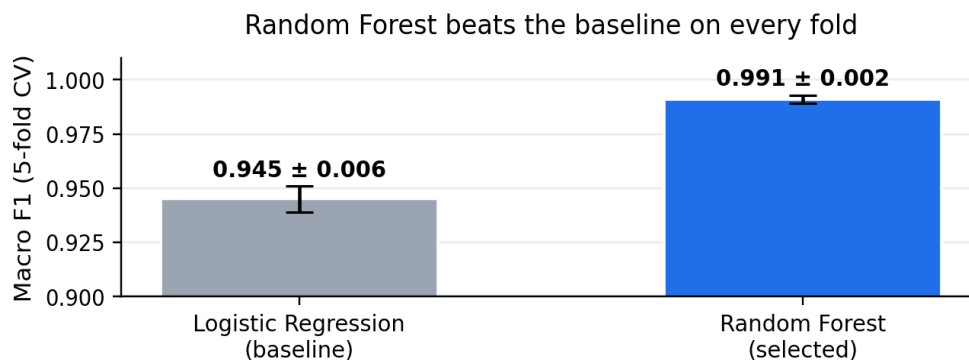


Figure 1. Model comparison. Error bars are one CV standard deviation.

On the held-out test set, the statistic I care about most is how the Random Forest handles the High tier, because a missed High is the exact outcome the stakeholder is paying to prevent. Of the 356 true-High developers in the test set, the model correctly flagged 351 (98.6% recall). The five it missed were all classified as Medium, not Low. That matters: every missed High still triggers at least a scheduled check-in under the recommended-action mapping. A two-step miss (High getting labeled Low) would be a wellness failure. A one-step miss (High getting labeled Medium) still produces a workload conversation, even if on a slower timeline than the stakeholder would want.

Per-tier performance is pretty consistent. F1 for Low, Medium, and High all land above 0.98 on the held-out test set, which tells me the model isn't getting its headline macro F1 number by trading off minority-class performance for wins on the majority class. Precision on High is slightly below recall on High (a few Medium cases are getting flagged as High), which is actually the direction of asymmetry you want for a triage tool. A false High costs you a 15-minute conversation. A false Low costs you an attrition event.

Feature importance (Figure 2) is where the headline number needs an asterisk. The self-reported stress_level feature accounts for roughly 70% of the Random Forest's total Gini importance. The next seven features combined contribute less than it does on its own. That matters for two reasons. First, stress_level correlates 0.60, 0.55, and 0.49 with daily_work_hours, screen_time, and bugs_per_day respectively, so the behavioral features aren't independent sources of signal. They're close proxies for whatever stress_level is already capturing. Second, the target itself was derived from a continuous Burn Rate that's a self-reported measure of basically the same underlying state. So there's a real risk the model is just restating stress rather than predicting independent risk from behavior. Gini importance is a predictive decomposition of variance inside the trained model anyway, so none of this should be read causally. The caveat in the chart isn't decorative.

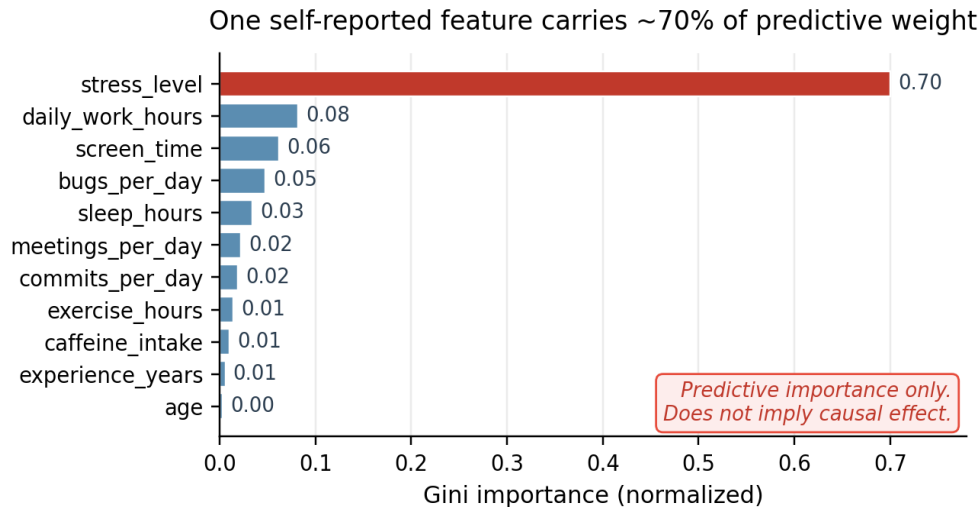


Figure 2. Random Forest Gini feature importance, sorted. Red bar = dominant self-report feature.

5. Recommendation

Situation. Engineering team leads need to triage wellness check-ins across their reports before attrition forces the issue. The process right now is reactive and leans on symptoms that only get visible once burnout is already advanced. A model that accurately flags High-tier developers would move that decision upstream.

Complication. The Random Forest delivers macro F1 = 0.991 ± 0.002 in 5-fold CV, and 98.6% recall on held-out High cases with zero two-step misclassifications. On standard ML evaluation criteria, that's a win. But 99% macro F1 on a human behavioral outcome is close to implausible in real-world deployment. Combined with the uniform 2% missingness and zero Tukey outliers, the most likely explanation is that the training data is synthetic or heavily pre-cleaned. The other concern is that a single self-reported feature is doing most of the predictive work, and the target itself is derived from a self-report of the same construct. Behavioral features are pulling their weight, but only modestly. Both of those things limit how far the performance number can be trusted outside the training distribution.

Resolution. Deploy the Random Forest as a weekly screening aid through the Streamlit dashboard, with human review in the loop. Three rules are non-negotiable. (1) The predicted tier is advisory input to a team-lead conversation. It is never a performance-review or compensation signal. (2) The dashboard's built-in low-margin warning, which triggers when the top two class probabilities are within 15 percentage points, is treated as a required escalation to human judgment, not a soft suggestion. (3) The tool gets reviewed for disparate impact before any expansion beyond a single pilot team. There are also three next steps I'd want to do before putting real weight on the model. First, retrain without stress_level and compare CV performance directly. If the behavior-only version is meaningfully worse, that confirms my concern about stress restatement, and it means the tool only has a defensible use case where self-reports are reliably collected. If the behavior-only version holds up, the tool gets a useful fallback mode for when self-reports are missing or gamed. Second, validate on internal company data before any production decision. CV on a Kaggle dataset doesn't tell you real-world generalization. Third, extend the feature set to include longitudinal signals: four-week trends in commit cadence, sleep, and meeting load instead of just point-in-time snapshots. Burnout is a trajectory, and a model that can't see trajectories is missing the most useful signal the stakeholder actually cares about.

Uncertainty and scope. The headline performance number is macro $F1 = 0.991 \pm 0.002$ on 5-fold CV within this dataset. The real-world confidence interval on any specific engineering organization is unknown, and it's almost certainly wider than the CV interval suggests. My recommendation is therefore to adopt the tool for screening only, after an internal-data validation pilot, with the understanding that predicted tiers aren't causal claims about any individual developer and should never drive adverse employment decisions.

Operating procedure. In a production pilot, the intended weekly flow would work like this. At the start of each cycle, the team lead opens the dashboard and enters current work-pattern and self-report values for each report. Any developer flagged Medium or High gets reviewed. Low-margin cases get escalated to manual review. Confident High cases trigger a same-week wellness conversation. Confident Medium cases go into a two-week check-in queue. The team lead also logs whatever intervention (if any) happens and what the outcome looked like afterward. That logged ground truth is what eventually lets you turn a Kaggle-trained classifier into a model that's actually tuned to your specific engineering org.

Threats to validity

Pulling the caveats I've raised throughout the report into one place, there are four threats to validity worth calling out explicitly before anyone makes a decision about adopting this tool. For each one, I note what the evidence is and what the mitigation path looks like.

Threat	Evidence	Mitigation
Stress restatement	stress_level carries ~70% of Gini importance; correlates 0.60 with daily work hours	Refit without stress_level and compare CV performance directly
Likely synthetic source data	Uniform 2% missingness across all columns, zero Tukey outliers, 99% macro F1	Validate on internal company data before any production decision
Tier cutoff arbitrariness	Low / Medium / High are analyst-set percentile cuts on a continuous Burn Rate	Sensitivity-test alternative cutoffs; use the continuous score where stakes allow
Point-in-time features	The model sees a single snapshot, but burnout is fundamentally a trajectory	Add rolling four-week trend features before any pilot expansion

Table 2. Threats to validity with mitigation paths. Rows one and two are the binding constraints on any real-world deployment decision. Rows three and four are second-order.